

- Market Anomaly Detection System — Demo Script
 - Before Recording
 - Scene 1 — The Cost of Getting It Wrong (1.5 min)
 - Scene 2 — The Command Centre (1 min)
 - Scene 3 — Prove It's Live (3 min) ★ THE CENTREPIECE
 - 3a — Normal Transaction First
 - 3b — Suspicious Transaction
 - 3c — Velocity Attack Pattern
 - 3d — The Proof: Custom Transaction
 - Scene 4 — How Analysts Work With It (2 min)
 - 4a — Triage
 - 4b — Investigate
 - Scene 5 — The Feedback Loop (1 min)
 - Scene 6 — Model Performance (1 min)
 - Scene 6b — Settings & API Keys (30 sec)
 - Scene 7 — How It Works Under the Hood (2 min)
 - Scene 8 — Close (30 sec)
 - Quick Reference
 - If He Asks

Market Anomaly Detection System — Demo Script

For: Jonathan Maharaj FCPA — Strategic Finance Advisor **Format:** Screen-recorded walkthrough (~12–15 min) **Rule:** All amounts in ₹ INR. Show, don't tell. Prove it's live.

Before Recording

```
# Terminal 1 — Backend
cd backend && source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2 — Frontend
npm run dev # → http://localhost:3000
```

Quick check: `curl http://localhost:8000/health → {"status":"healthy"}`

Scene 1 — The Cost of Getting It Wrong (1.5 min)

Don't open the app yet. Look at the camera and set the stakes.

Say this:

"In 2024, global fraud losses crossed **48billion** *

* *.But there's the number nobody talks about — for every dollar lost to actual fraud, financial institutions spend*

* **4.41** in investigation, compliance, and operational overhead trying to catch it. That's the real cost.

And the irony? Most of that spend is wasted chasing **false positives**. The average bank fraud team processes 200,000+ alerts per year. Industry data shows 80–95% of those are false alarms — legitimate customers flagged by dumb rules. Every false positive costs \$50–150 to investigate. Multiply that out, and you're burning millions on noise.

Meanwhile, the actual fraud slips through. Why? Because traditional systems are **rule-based** — hard-coded thresholds like 'flag anything above ₹5 lakhs' or 'block transactions from this country.' These rules are written by humans, updated quarterly if you're lucky, and completely static. Fraudsters reverse-engineer them in days.

There's a second problem no one talks about: **analyst fatigue**. When 9 out of 10 alerts are false, your best investigators start rubber-stamping. They stop reading the details. And that's exactly when the real fraud — the ₹2 crore wire transfer at 3 AM to a brand-new account — gets marked 'legitimate' because someone was drowning in garbage alerts.

So I built something fundamentally different. Instead of rules that ask 'does this transaction break a threshold?', this system asks '**does this transaction look normal for this specific entity, at this time, through this channel, to this destination?**' It learns behavioral baselines per entity and flags genuine deviations — not arbitrary limits.

Three detection layers. Real-time scoring in under 25 milliseconds. A closed feedback loop where every analyst decision makes the next detection smarter. Let me show you."

Scene 2 — The Command Centre (1 min)

Open <http://localhost:3000>. Dashboard loads.

Walk through:

- Point to **4 KPI cards** at the top: Total Transactions (2,306), Active Alerts (362), High-Risk (162), False Positive Rate (23.3% ↓)
- Point to the **severity donut** — 104 CRITICAL, 186 HIGH
- Glance at the **alerts trend chart** — highlight any spike

Say this:

"This is the operations dashboard. The system has processed over 2,300 transactions. Out of those, 362 are flagged as active alerts — 104 of which are critical, meaning all three detection layers agree something is wrong.

See that false positive rate — 23.3% and trending down. That means roughly 3 out of 4 alerts we raise are genuine. In a traditional system, that number is often inverted — 70–80% false positives. The difference is the feedback loop, which I'll show you in a moment."

Scene 3 — Prove It's Live (3 min) ★ THE CENTREPIECE

Scroll down to the **Live Transaction Scanner** on the Dashboard page. This is where you prove the ML engine is real.

3a — Normal Transaction First

Select **Preset Scenarios** tab → choose  **Normal: ₹2,850 grocery purchase** → click **Scan Now**

Say this:

"Let me score a transaction live. ₹2,850 at a grocery store — regular POS channel, business hours, same store the account has visited before. Watch..."

(result appears — Score ~18, LOW, No Alert)

"Score: 18, LOW severity. No alert raised. The model recognises this as normal spending behaviour. Processed in under 10 milliseconds."

3b — Suspicious Transaction

Select  **Suspicious: ₹76.7L at 2 AM, unknown IP** → click **Scan Now**

Say this:

"Now the same engine, same moment — but ₹76.7 lakhs wired at 2 AM to an account this entity has never transacted with, from an unknown IP, via API instead of their usual web channel."

(result appears — Score ~90, CRITICAL, Alert Created)

"Score: 90, CRITICAL. All three detectors lit up. The statistical detector flags the amount — 2,400 standard deviations above normal. The behavioral detector spots the new destination, new channel, and unusual hour. The ML model — Isolation Forest — catches the multi-feature anomaly that no single rule would see. And look — every flag has a plain-English explanation. That's the audit trail compliance teams need."

3c — Velocity Attack Pattern

Select  **Velocity spike: 6 rapid ₹15L wire transfers** → click **Scan Now**

Say this:

"One more — six rapid-fire wire transfers of ₹15 lakhs each, 40 minutes, to six different accounts. This is the signature of account takeover fraud."

(result appears — Score ~56, MEDIUM, Alert Created)

"Score: 56, MEDIUM — alert raised. The behavioral detector catches the frequency spike — far above this entity's baseline. It's not as severe as the 2 AM scenario because the individual transfer amounts aren't as far outside the entity's pattern — a correct nuance. In a rule-based system, this would either be missed entirely or scored identically. Here, the model distinguishes severity automatically."

3d — The Proof: Custom Transaction

Click **Custom Transaction** tab. This is the moment that removes all doubt.

Demo sequence:

1. Type **5000** in Amount, select **WEB**, hour **14** → **Scan Now** → Score ~42 LOW
2. Change amount to **5000000** → **Scan Now** → Score jumps to ~71 HIGH
3. Change hour to **3** → **Scan Now** → Score rises to ~81 HIGH (unusual hour triggers)
4. Change channel to **API** → **Scan Now** → Score climbs to ~86 HIGH (new channel triggers)

Say this:

"Now let me prove this isn't pre-recorded. I'm going to type a completely new transaction — ₹5,000, web channel, 2 PM. Score 42, LOW — as expected.

Now I change the amount to ₹50 lakhs — watch the score jump to 71, HIGH. The z-score alone is massive — this entity normally transacts around ₹18,000.

I move the hour to 3 AM — it climbs to 81. Now I switch the channel from WEB to API — 86.

Every change I make produces a different score because the Isolation Forest model is evaluating across all three detector layers in real-time. Same pipeline, same code, completely different inputs — completely different outputs. This is live ML scoring right now."

Scene 4 — How Analysts Work With It (2 min)

4a — Triage

Click **Alerts** in the sidebar.

Say this:

"An analyst starts their day here. 650 alerts — they don't scroll through all of them."

Filter by **Severity** → **CRITICAL**, then **Status** → **ACTIVE**.

"CRITICAL plus ACTIVE. These are the fires. 104 cases where all three detectors agreed. The analyst picks the highest risk score and clicks in."

4b — Investigate

Click **Investigations** in the sidebar. Type **ALT-06353** → Load.

Walk through the screen:

- Transaction: **TXN-78582**, ₹67,034 via POS
- Risk score: 93.58 — CRITICAL
- IP: **209.212.82.74**, device fingerprint: **41f60367-6adc-4d**
- Detector breakdown: statistical 15.67, behavioral 59.87, ML 13.02

Say this:

"Everything the analyst needs in one screen. ₹67,034 POS transaction, risk score 93.58 — critical. IP address, device fingerprint, channel, the detector score breakdown — all here. No switching between five different systems."

Type in notes: "**Geo-IP mismatch: account registered in India, transaction IP resolves to US data centre. Escalating.**" → Click **Add Note**.

Then click **Mark as Fraud**.

"30 seconds — note added, decision submitted, case closed. That decision is now stored in the database as a labelled training example. Watch what happens next."

Scene 5 — The Feedback Loop (1 min)

Click **Feedback History** in the sidebar.

Say this:

"The decision I just made is now part of the feedback pool. 83 labelled decisions so far — 55 confirmed frauds, 27 false positives. Each one carries the analyst name, timestamp, and notes — fully auditable.

Here's why this matters: these labels are what the model retrains on. Every decision an analyst makes doesn't just close a case — it makes the next detection more accurate. That's the feedback loop. It's why the false positive rate trends down over time rather than staying flat."

Scene 6 — Model Performance (1 min)

Click **Analytics** in the sidebar.

Walk through:

- Precision: 67%, Recall: 35%, F1: 46%
- Alert volume trend chart
- Model version history

Say this:

"Precision 67% — two-thirds of our alerts are real fraud. For an unsupervised model with no labelled data at launch, that's strong. Recall is 35% — plenty of room to improve, and each batch of analyst feedback pushes it higher.

We track this per model version so if a retrained model regresses — we see it immediately before it hits production."

Scene 6b — Settings & API Keys (30 sec)

Click **Settings** in the sidebar. Scroll to the **API Keys** card.

Say this:

"One thing worth showing — the Settings page has API key management. In production, this is how external systems integrate with the detection engine. Your core banking platform, a SIEM tool like Splunk, or an ERP system can send transactions to the scoring API using this key. Every call is authenticated and rate-limited.

The alert thresholds above are also configurable per deployment — what counts as HIGH or CRITICAL can be tuned to match your institution's risk appetite without touching the model."

Scene 7 — How It Works Under the Hood (2 min)

This can be spoken over the Analytics page, or you can briefly show the architecture diagram.

Say this:

"Quick architecture overview. The frontend is React with Next.js — what you've been looking at. Behind it, a FastAPI Python backend handles all the API calls asynchronously. Every transaction is stored in PostgreSQL.

The detection engine has three layers:

One — a statistical detector. Z-scores, high-value thresholds, velocity rules. This catches the obvious outliers — amounts that are 10x normal, rapid-fire transfers. It carries 25% of the total score weight.

Two — a behavioral detector. This compares every transaction against that specific entity's history. New destination? New channel? Activity after a dormant period? Different time pattern? 35% weight.

Three — an Isolation Forest ML model. This is unsupervised — it doesn't need labelled fraud data to train. It learns the shape of normal across multiple features simultaneously and flags whatever doesn't fit. 40% weight.

The three scores are combined into one composite risk score. Above 50 triggers an alert. Above 90 is critical. And the feedback loop from analyst decisions feeds back into retraining.

The whole scoring pipeline runs in under 25 milliseconds per transaction. At production scale with multiple workers, that's 500,000+ transactions per day."

Scene 8 — Close (30 sec)

Say this:

"So to summarise — this isn't a monitoring dashboard on top of static rules. It's an end-to-end ML detection platform: real-time scoring, explainable alerts, a full investigation workflow, and a closed feedback loop that makes the model smarter with every analyst decision.

Happy to share the repository, walk through the deployment architecture, or discuss how this could be adapted for a specific use case. Thanks for watching."

Quick Reference

Metric	Value
Transactions	2,306
Total Alerts	650 (104 CRITICAL)
Active Alerts	362
False Positive Rate	23.3% ↓
Feedback Pool	83 decisions
Precision / Recall / F1	67% / 35% / 46%
Detection Speed	< 25 ms
Investigation Alert	ALT-06353 (score 93.58)

If He Asks

Question	Answer
Is the data real?	The database has representative financial transactions to show the system at scale. The ML engine, API, and workflow are all running live — as I demonstrated with the custom scanner. In production, this connects to a live transaction feed via Kafka or direct API ingestion.
How does it scale?	FastAPI is async — non-blocking IO. Add Uvicorn workers for CPU parallelism. PostgreSQL read replicas for query load. Celery + Redis queue in front of the detection engine for traffic spikes.
Supervised or unsupervised?	Isolation Forest is unsupervised at launch — no labelled data needed. The feedback loop enables supervised fine-tuning over time.
How do you prevent model	Analytics page tracks precision/recall per model version. Degradation is visible before it hits production. Canary rollout for new versions.

Question	Answer
drift?	
Deployment?	AWS ECS (containerised FastAPI) + RDS PostgreSQL (Multi-AZ) + CloudFront/Vercel for frontend.
Why not just use rules?	Rules are static — fraudsters adapt. The ML layer catches multi-feature anomalies no single rule would flag. The behavioral layer personalises detection per entity. Rules alone give you 70%+ false positive rates.
Data privacy?	No unnecessary PII. Device fingerprints truncated. Audit trail append-only. CORS locked.
What's the API key for?	External systems (core banking, SIEM, ERP) authenticate against the detection API using this key. In production, every <code>/api/detection/evaluate</code> call requires a valid key in the <code>Authorization</code> header. Keys are scoped per integration and can be revoked independently.